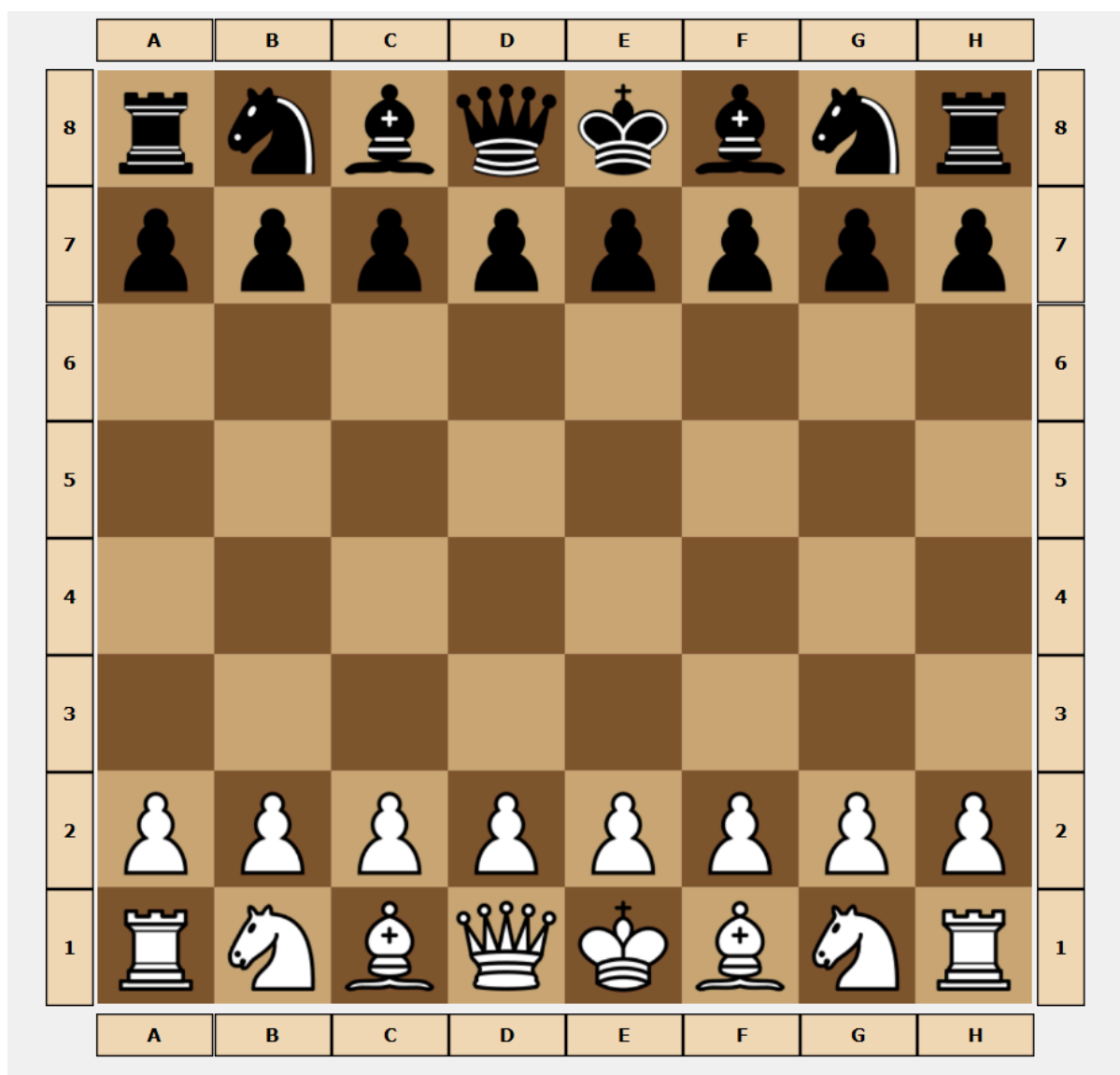


Relatório do Trabalho Prático de Programação Avançada 2024/25



Membros do grupo:

- Artur Silvano Capelossi - 2023143204
- Diogo António Mendes Beja - 2023135260
- Nikolay Denisovitch Grachev - 2023142365

Turma Prática: P5

Objetivo do trabalho

O objetivo deste trabalho foi desenvolver uma aplicação em Java que simule um jogo de xadrez, utilizando também JavaFX para a interface gráfica

Decisões de Design e Implementação

No desenvolvimento do projeto, seguimos princípios de boa organização e estruturação do código para garantir clareza e manutenção simple:

- **Separação entre a interface do utilizador e o modelo de dados:**

Esta separação permitiu uma melhor organização do código, facilitando a reutilização e a manutenção.

- **Utilização de uma Facade como gestor do modelo de dados:**

Implementámos uma classe Facade (ChessGameManager) que atua como ponto de acesso único ao modelo, promovendo o encapsulamento e simplificando a comunicação entre a interface e a lógica do jogo.

- **Mecanismo de notificação com PropertyChangeSupport:**

A interação entre a UI e o modelo foi baseada no padrão Observer, utilizando PropertyChangeSupport. Este mecanismo permite que as views sejam atualizadas automaticamente sempre que ocorrem alterações relevantes no estado do modelo.

- **Padrão Singleton no ModelLog:**

A classe ModelLog foi implementada como um Singleton para garantir que exista apenas uma instância responsável pelo registo de eventos e ações ao longo da execução do jogo.

- **Serialização para persistência de jogos:**

A funcionalidade de guardar e carregar o estado completo do jogo foi implementada através de serialização, proporcionando uma forma simples e eficaz de persistir os dados entre sessões.

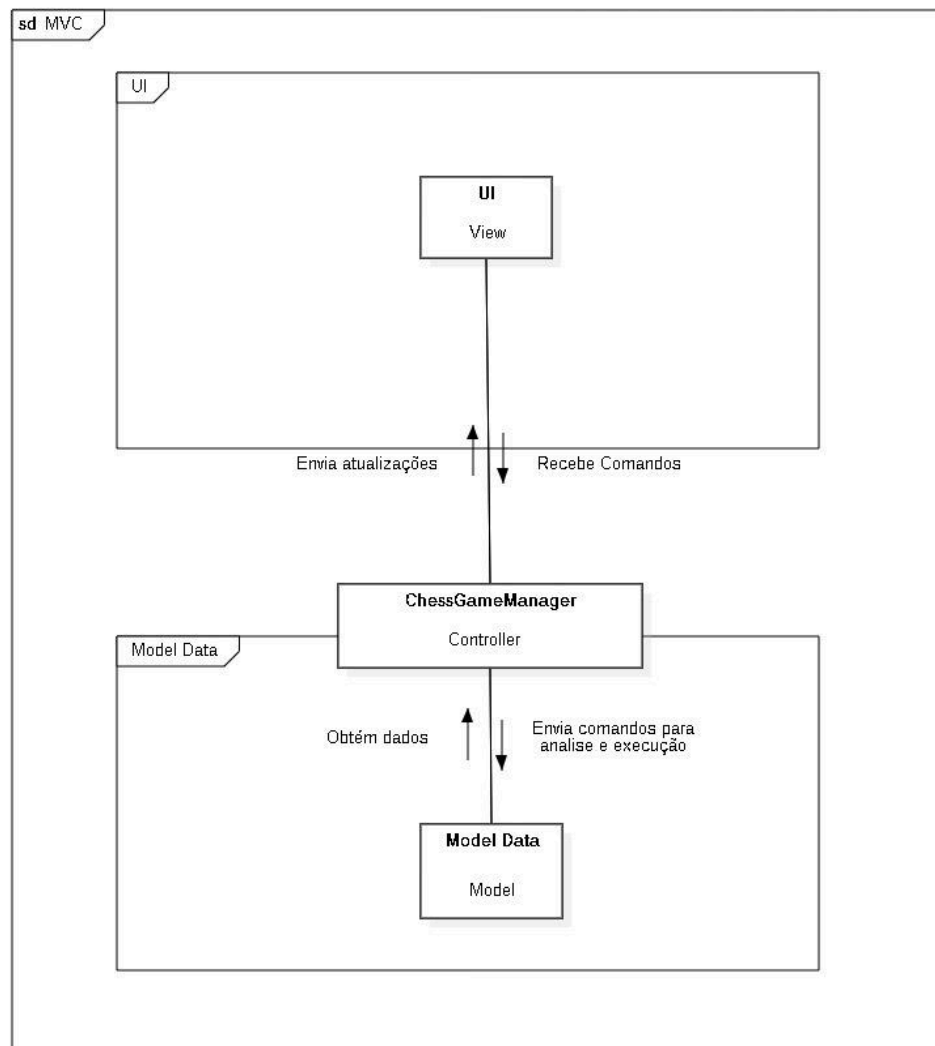
- **Padrão Command para controlo de ações:**

Aplicámos o padrão Command para encapsular ações do jogo como objetos, o que permitiu implementar funcionalidades como undo e redo de forma limpa, modular e extensível.

Design Patterns aplicados no projeto

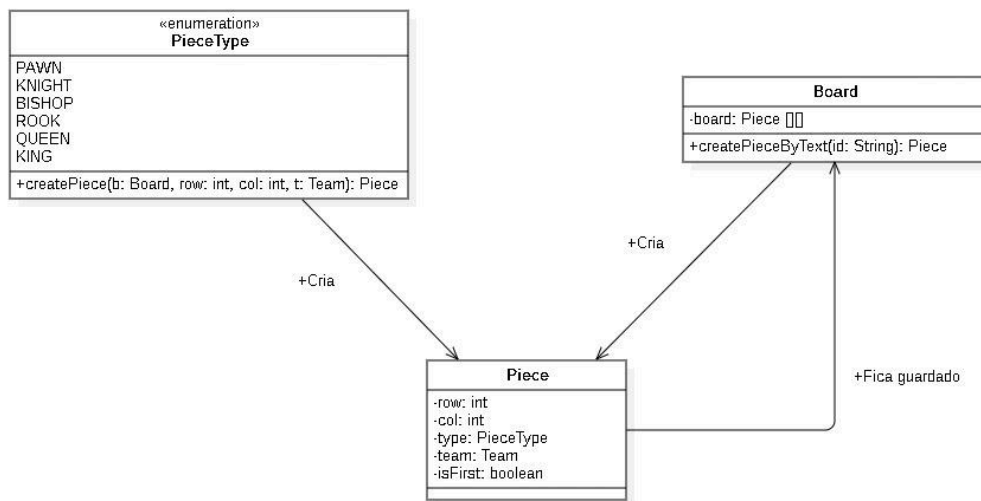
- **MVC (Model - View - Controller):** O model é o Data Model (modelo de dados), a view é a UI (User Interface) e o controller é o responsável por conectar e partilhar informações entre os dois da melhor forma, no caso é o ChessGameManager, que

recebe os comandos do usuário e os encaminha para o modelo de dados, onde é analisado/executado com base na lógica do jogo. Depois a UI é avisada que o modelo de dados foi alterado e então atualiza.

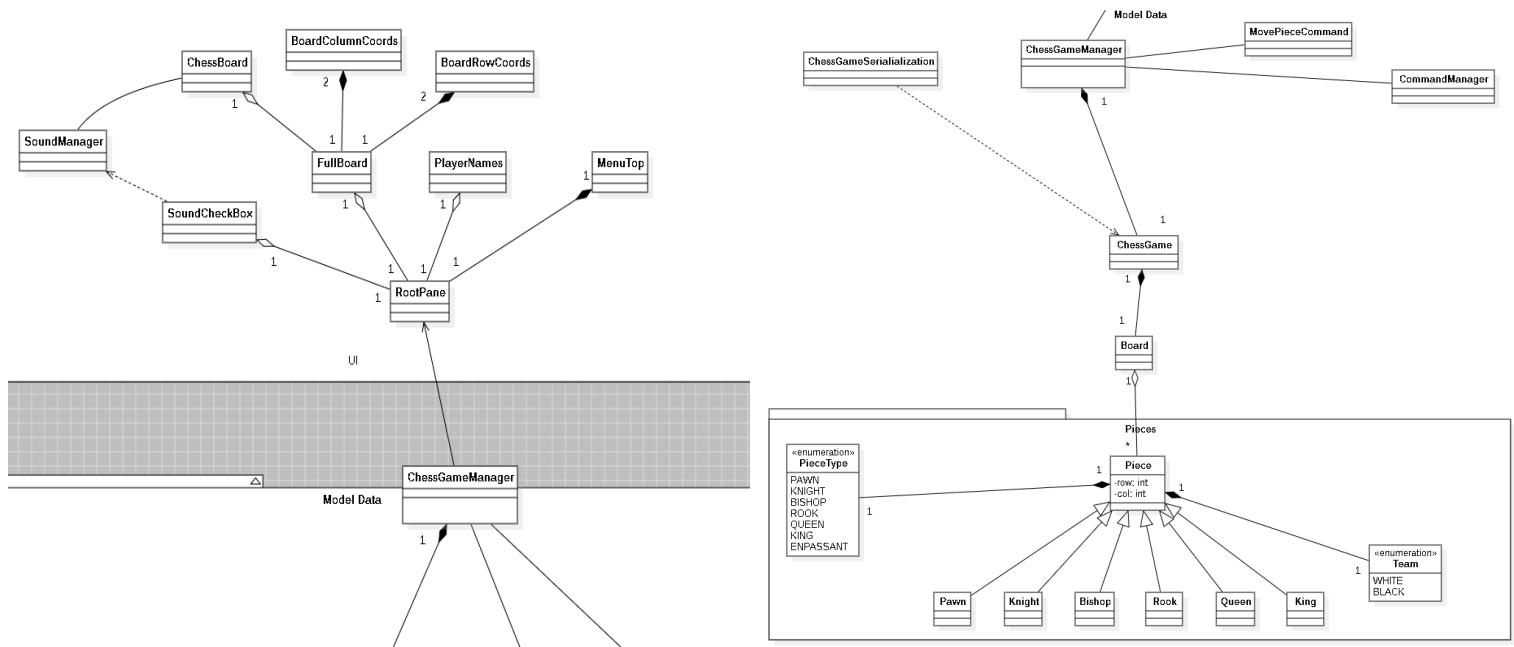


- **Factory method:** O factory method foi utilizado em um método de criação de peças dentro da enum PieceType, createPiece(), que com base no tipo que chamou o método, uma nova Piece do mesmo PieceType era criada. O outro método foi o de criação de peças baseado numa string, composta por 3 ou 4 caracteres, em que o primeiro define o tipo (dependendo da letra) e a equipa (se for maiuscula ou minuscula), o segundo a coluna que está na Board, o terceiro a linha que está na Board e se tiver um quarto (normalmente um "*"), quer dizer que ainda não realizou

nenhum movimento.

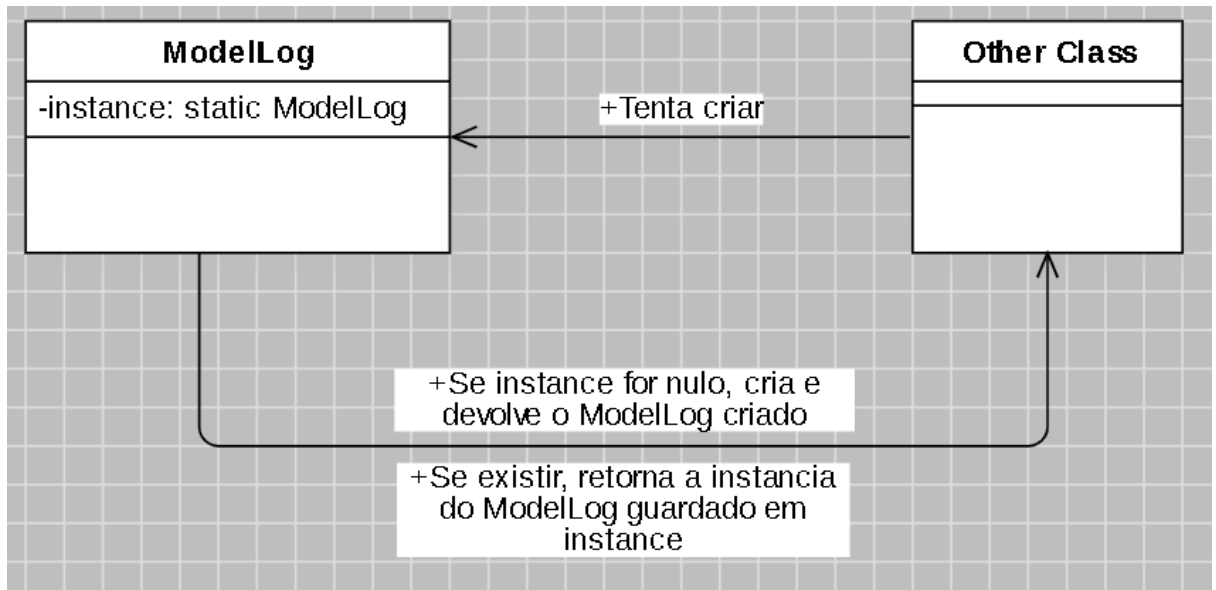


- **Facade:** O `ChessGameManager` é uma facade do projeto, pois esconde e intermedia o acesso da UI aos dados do jogo, e cria uma interface simplificada para que as classes da UI consigam acessar todas as informações que precisam através de apenas uma classe. Outra facade do projeto é a classe `ChessGame`, que esconde as classes mais básicas do jogo, `Board`, `Piece` e suas subclasses para impedir que elas tenham qualquer contato direto com o resto do código, pois poderiam ser facilmente alteradas, causando problemas no jogo.

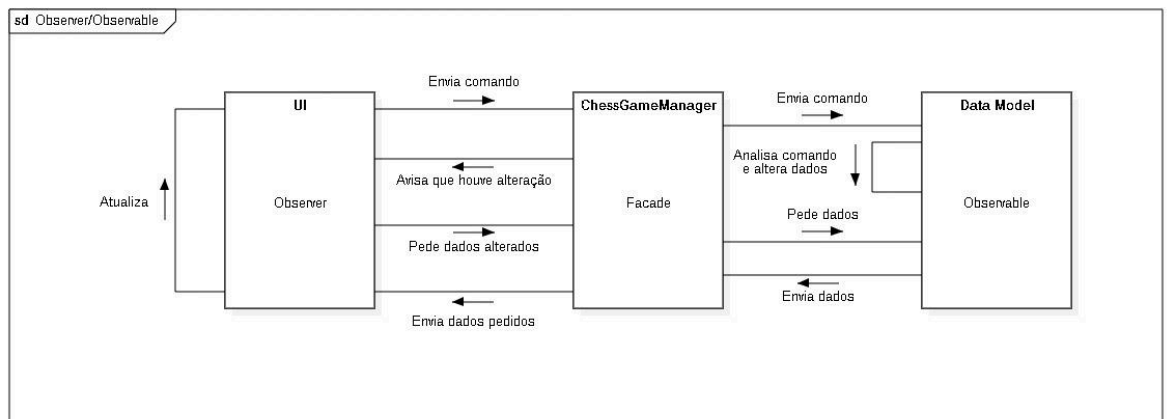


- **Singleton:** É usado na classe `ModelLog`, para que haja apenas uma do mesmo tipo, pois no caso não faz sentido que haja múltiplas iguais. Na tentativa de criar uma

nova instância, a primeira criada é retornada, e portanto são a mesma.

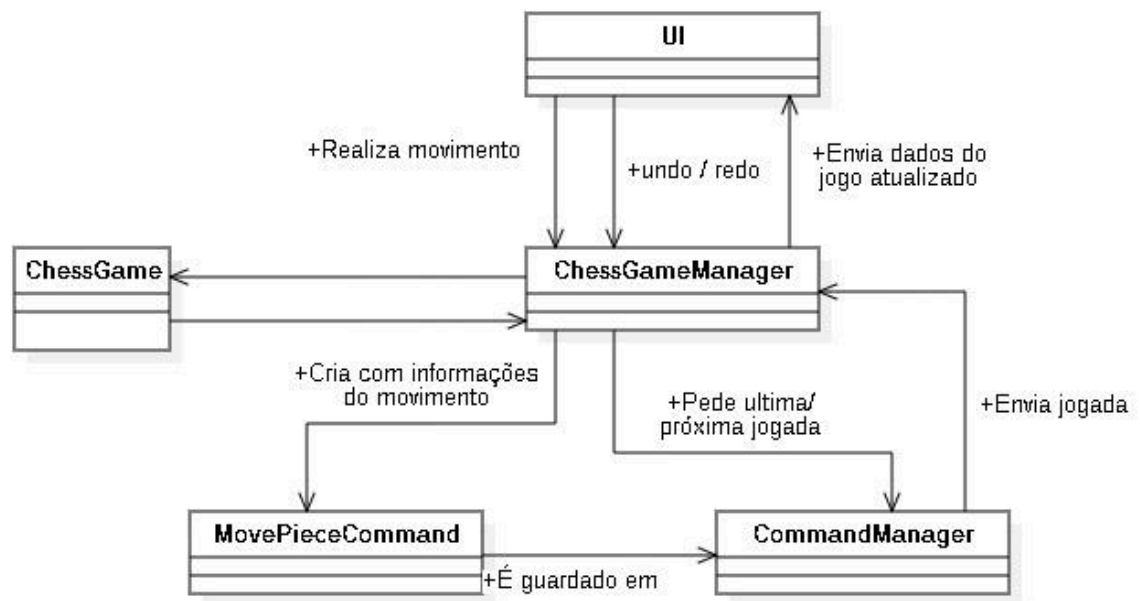


- **Observer/Observable:** No caso do projeto, os Observers são os componentes da UI, que estão sempre à espera que o modelo de dados seja alterado para que atualize a si próprio, e o Observable é todo o Data Model, mais especificamente a facade, já que toda as interações e, consequentemente, mudanças passam por ele.



- **Command Pattern:** (CommandManager e MovePieceCommand) Cada movimento realizado no jogo (MovePieceCommand) é guardado no CommandManager numa ArrayDeque de MovePieceCommand, o que torna possível os comandos de undo e

redo.



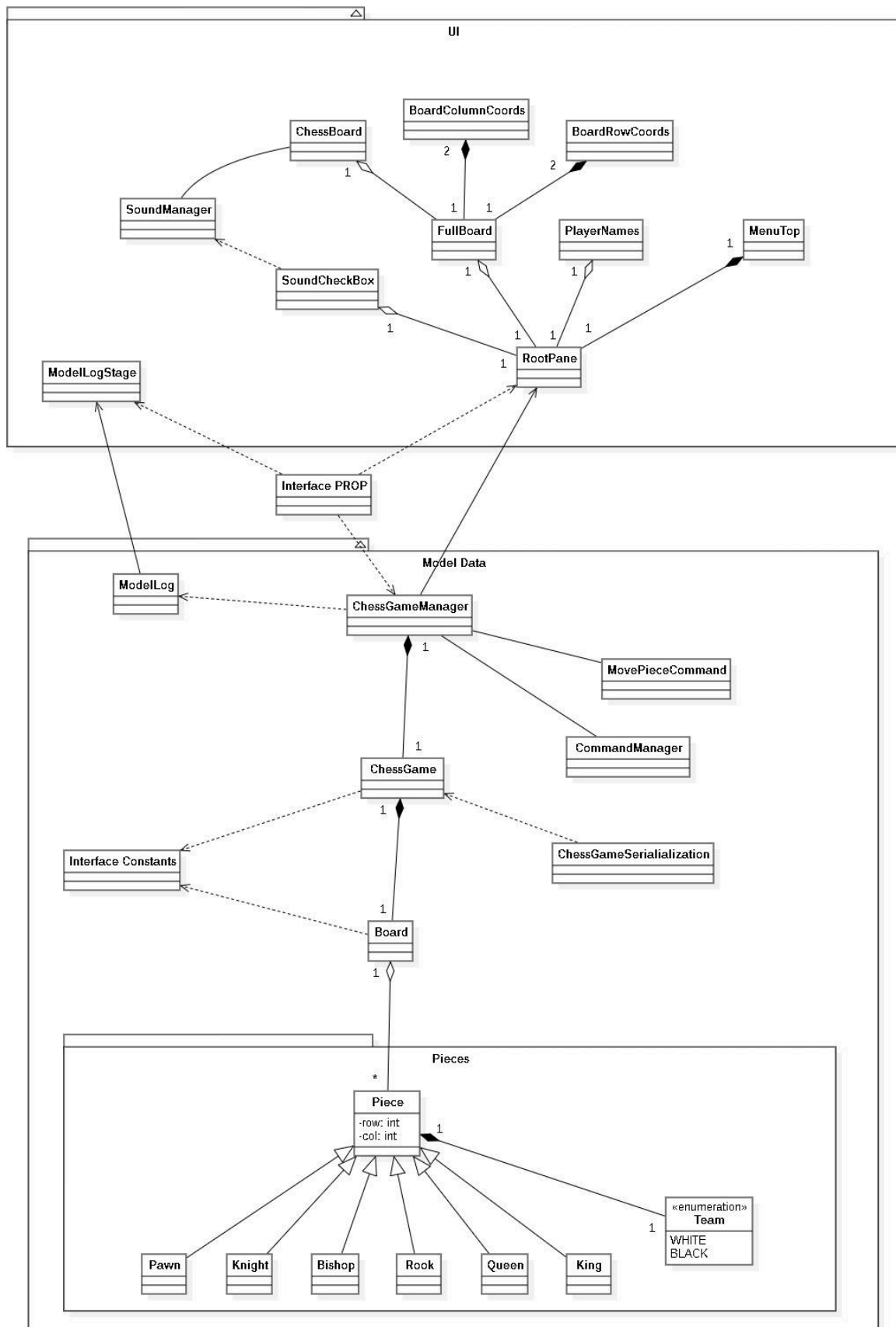
Descrição das classes

Classe	O que representa	Objetivo
Board	Representa o tabuleiro de xadrez	Modelar a estrutura do tabuleiro de xadrez, inicializar a disposição das peças no início do jogo, gerir a movimentação das peças, fornecer métodos de consulta ao estado do tabuleiro e gerar uma representação textual do seu estado atual, incluindo as possíveis jogadas de uma peça.
Piece	Representa uma peça abstrata do xadrez	Definir estrutura e comportamentos base de todas as peças de xadrez, incluindo posição, equipa, tipo e lógica comum de movimentação, segurança do rei e captura.
PieceType	Enumeração dos tipos de peças de xadrez (PAWN, KNIGHT, BISHOP, ROOK, QUEEN, KING, ENPASSANT)	Definir os diferentes tipos de peças e fornecer um método para criar instâncias concretas de cada tipo, exceto ENPASSANT, que é especial.
Team	Enumeração para as duas equipas (WHITE, BLACK)	Representar as duas equipas do jogo: brancas e pretas. Utilizada para identificar a cor de cada peça.

Classe	O que representa	Objetivo
Bishop, King, Knight, Pawn, Queen, Rook	Representam cada uma um tipo diferente de peça	Define a movimentação de cada peça e verifica se têm alguma movimentação especial, implementando-a se existir
enPassant	Representa uma peça especial e temporária do tipo ENPASSANT	Peça temporária usada para sinalizar a possibilidade do movimento especial "en passant" após um peão avançar duas casas. Não realiza movimentos e é removida após um turno.
ChessGame	Modelo de dados do jogo	Gerencia o estado interno do jogo: jogadores, tabuleiro, turno, seleção de peças, verificação de fim de jogo, xeque, e jogadas possíveis.
ChessGameManager	Facade que serve como controlador principal do jogo de xadrez	Atua como um controlador no padrão MVC: intermédio entre o modelo de dados ChessGame e a UI, executa comandos (com suporte a undo/redo), trata eventos e exporta/importa o jogo.
ChessGameSerialization	Classe para serialização e desserialização do jogo	Fornecer métodos estáticos para salvar (serialize) e carregar (deserialize) o objeto ChessGame em arquivos, permitindo a persistência e recuperação de partidas em andamento.
ChessMain	Main da aplicação	Lançar a aplicação
Constants	Interface de constantes globais usadas no jogo de xadrez	Armazenar valores fixos como o tamanho do tabuleiro e as letras do eixo X. Fornece também um método para converter letras em índices numéricos de coluna.
ModelLogs	Registro de logs do jogo (Singleton)	Armazenar e notificar mensagens de log relacionadas ao andamento do jogo (como movimentos, importações, exportações).
CommandManager	Gerenciador de comandos do jogo (padrão Command)	Controlar a execução, fazer undo e redo de comandos realizados no jogo, armazenando o histórico.
ICommand	Interface para comandos executáveis	Definir uma estrutura comum para comandos com métodos para executar e desfazer ações, seguindo o padrão <i>Command</i> .
MovePieceCommand	Um comando para movimentar uma peça no tabuleiro	Executar e desfazer uma jogada no xadrez, incluindo movimentos normais, captura, en passant e roque,, armazenando todas as informações necessárias para reverter a jogada.

Classe	O que representa	Objetivo
PROP	Interface com nomes de eventos	Centralizar os nomes das propriedades usadas para notificar mudanças de estado no modelo, facilitando a comunicação entre modelo e interface.
BoardColumnCoords	Componente visual da interface com coordenadas das colunas do tabuleiro	Mostrar as letras acima do tabuleiro de xadrez, ajustando dinamicamente o tamanho conforme o tabuleiro muda.
BoardRowCoords	Componente visual da interface com coordenadas das linhas do tabuleiro	Mostrar os números ao lado do tabuleiro de xadrez, ajustando dinamicamente o tamanho conforme o tabuleiro muda.
ChessBoard	Canvas principal do tabuleiro de xadrez	Desenhar o tabuleiro, mostrar as peças, lidar com cliques do jogador e reproduzir sons durante o jogo.
FullBoard	Componente gráfico completo do tabuleiro de xadrez	Agrupar o tabuleiro, as coordenadas das colunas e linhas, e ajustar dinamicamente o tamanho.
MenuTop	Barra de menu do jogo (topo da janela)	Fornecer opções para iniciar, carregar, salvar, importar, exportar ou sair do jogo, além de funcionalidades como undo/redo e exibir jogadas possíveis.
ModelLogStage	Janela separada para exibir logs	Exibir e permitir limpar o histórico de ações realizadas no modelo (ModelLog).
PlayerNames	Caixa vertical com nomes dos jogadores	Mostrar os nomes dos dois jogadores e indicar de quem é a vez no jogo.
RootPane	Janela principal da aplicação	Organizar e exibir todos os elementos da interface do jogo de xadrez.
SoundCheckBox	Caixa de seleção de som	Permitir ao utilizador ativar ou desativar os sons do jogo de xadrez.
ImageManager	Gerenciador de imagens utilizadas no jogo	Carregar e fornecer imagens a partir da pasta images/, armazenando-as em cache para evitar múltiplas leituras do disco e melhorar o desempenho.
SoundManager	Gerenciador de sons da aplicação de xadrez	Controlar a reprodução de sons (ex: movimentos, capturas) com suporte a fila, autoplay e parada. Carrega sons da pasta sounds/ e evita sobreposição.
ChessUI	Classe principal da interface gráfica (JavaFX)	Inicializa e exibe a interface gráfica do jogo de xadrez com múltiplas janelas, usando ChessGameManager como controlador e RootPane como conteúdo principal.

Relações entre as classes



Model Data

- **Pawn, Knight, Bishop, Rook, Queen e King** são todas subclasses de **Piece**, que possui todas as características gerais de uma peça de xadrez, como **Team** (cor da peça) e posição (linha e coluna), enquanto as subclasses possuem todas as características e métodos da **Piece**, além da verificação de movimentos possíveis que elas podem fazer.
- A **Board** representa o tabuleiro de xadrez, portanto, contém as subclasses de **Piece** em uma array bidimensional. A **Board** contém métodos de adição e remoção de peças e criação de um novo tabuleiro com as peças em suas posições iniciais.
- O **ChessGame** guarda uma **Board**, e é onde realmente as regras e lógica do jogo são seguidas, é uma facade para a **Board** e as **Pieces**, que contém os métodos de seleção de peça, movimento, mudança de round, promoção e outros métodos importantes para o funcionamento do jogo.
- O **ChessGameManager** contém um **ChessGame**, e basicamente é a última camada do **ModelData** e não contém nenhuma grande lógica, tem basicamente métodos que levam informações da **UI** para o resto do **ModelData** e o mesmo ocorre ao contrário.
- **Constants** é uma interface que contém o número de células do tabuleiro, as letras que representam as coordenadas das colunas e um método que converte a coordenada da coluna (uma letra) para o número que a equivale. É usada para auxiliar na criação e navegação da **Board**.
- **ChessGameSerialization** é uma classe estática e contém métodos que fazem a serialização e deserialização de uma classe **ChessGame**, tornando possível importar e exportar essa classe.
- **CommandManager** e **MovePieceCommand** são as classes que tornam possível o funcionamento do undo e redo, elas são utilizadas em cada movimento realizado para salvar o estado do tabuleiro.
- **ModelLog** é uma classe que contém uma lista de Strings, em que são guardados os logs (cada ação realizada pelo user), é principalmente acessada pelo **ChessGameManager** que para cada ação cria um log no **ModelLog**.
- **PROP** é apenas uma interface onde são guardadas constantes usadas no **PropertyChangeSupport**, para a atualização dos elementos da **UI** quando ocorre alguma mudança relevante no **ModelData**

UI

- **RootPane** é a base do Stage (janela) principal
- **MenuTop** é o menu superior do **RootPane**, contém os botões para comandos essenciais do jogo, como a criação de jogo, import, export, save, open, além de comandos que auxiliam os jogadores, como os modos de jogo, mostrar movimentos possíveis e undo/redo. Por ter tantos comandos, tem acesso constante ao **ChessGameManager**.
- **PlayerNames** mostra basicamente quem são os jogadores e quem deve jogar no momento. Esses nomes são obtidos através do **ChessGameManager** e o jogador atual é atualizado em toda a mudança de round.

- **SoundCheckBox** é um checkbox que liga e desliga a assistência de som. Quando está ligada e uma ação é feita no jogo, o **SoundManager** controla os sons que descrevem o movimento feito, os arquivos de som para serem tocados são definidos na classe **ChessBoard**, com base nas respostas enviadas pelo **ModelData**.
- **FullBoard** é também contido pela **RootPane**, e basicamente contém as **BoardColumnCoords**, **BoardRowCoords** e o **ChessBoard**, não contém relação direta com o **ModelData**.
- **BoardColumnCoords** e **BoardRowCoords** também não tem relação com o **ModelData**, e servem para auxiliar os jogadores no entendimento e visualização das coordenadas do tabuleiro.
- **ChessBoard** é o principal elemento da **UI**, é onde o jogador seleciona as peças, faz o movimento e visualiza o estado atual do jogo. É a classe da **UI** que mais se relaciona com o **ModelData**, especificamente com o **ChessGameManager**.
- **ModelLogStage** é a janela em que é listado os logs do jogo, e em cada atualização do **ModelLog**, ele vai buscar os logs.

Features

Feature	Implementado	Parcialmente Implementado	Não Implementado
Iniciar um novo jogo (com entrada do nome do jogador)	x		
Salvar e carregar jogos usando serialização	x		
Importação (com entrada do nome do jogador) e exportação de jogos parciais no formato especificado	x		
Mecânica de jogo correta e aplicação de regras	x		
Deteção de fim de jogo (xeque-mate e empate)		x	
Movimentos especiais (roque, promoção de peão e en passant)	x		
Modo de aprendizagem com funcionalidades de undo e redo		x	
Descrições áudio de movimentos	x		

Deteção de fim de jogo (xeque-mate e empate):

- Xeque-mate
- Afogamento
- Regra das 50 jogadas
- Material Insuficiente

Modo de aprendizagem com funcionalidades de undo e redo: Num modo geral está implementado e a funcionar, porém o que não está 100% funcional é o redo para os movimentos especiais (en Passant, Castling e Promoção).